

INT315 · CLUSTER COMPUTING

UNIT V

Spark Streaming with Apache Kafka

Kafka Fundamentals, Cluster Architecture, Installation, Spark Integration & Ingestion Patterns

Theory -> Comic Visuals -> Industry Context -> Commands/Code -> Self-Assessment

Seven syllabus topics, in full depth, with heavy runnable code and seven original diagrams

Session 2026-27

Companion Study Material — INT315 Cluster Computing, Unit V

Table of Contents

Unit V at a Glance	4
1. Fundamentals of Apache Kafka.....	5
1.1 Events, Topics, and Partitions	5
1.2 Brokers, Retention, and Durability	6
1.3 Why Kafka, Specifically, for Streaming Data	6
2. Apache Kafka Cluster Architecture.....	8
2.1 Brokers and the Controller	8
2.2 Replication — Leaders and Followers	8
2.3 Consumer Groups and Cluster-Wide Coordination	9
3. Apache Kafka Installation	11
3.1 Prerequisites	11
3.2 Downloading and Extracting Kafka	11
3.3 Starting Kafka in KRaft Mode (No ZooKeeper Required)	11
3.4 Creating a Topic and Verifying the Installation	12
4. Integration of Apache Kafka with Spark	14
4.1 Reading From Kafka as a Streaming Source	14
4.2 Parsing Kafka's Raw Bytes Into Structured Data	15
4.3 Processing and Writing Results	15
4.4 Delivery Semantics — What Actually Happens on Failure	16
5. Producer & Consumer Messaging Model	19
5.1 Producers — Writing Events	19
5.2 Producer Acknowledgment Levels	20
5.3 Consumers and Consumer Groups	20
5.4 Committing Offsets — Manual vs. Automatic	21
6. Kafka Pipeline	23
6.1 Fan-In: Many Producers, One Topic	23
6.2 Fan-Out: One Topic, Many Independent Consumers	24
6.3 A Complete Pipeline, End to End	24
7. Ingestion Patterns	26
7.1 Direct Application Instrumentation	26
7.2 Change Data Capture (CDC)	26
7.3 Batch-to-Stream Bridging	27
7.4 Choosing an Ingestion Pattern	27

Unit V Review29

Unit V at a Glance

Unit IV closed the loop on batch and interactive analytics with Spark SQL. This unit opens the other half of Unit I's original batch-vs-real-time distinction properly: continuous, unbounded streams of events, ingested through Apache Kafka and processed by Spark Structured Streaming — the exact combination the fraud-detection and real-time-dashboard use cases from Unit I were built around, now covered end to end. Kafka is not part of Spark itself; it's an independent distributed messaging system that has become the de facto standard event-ingestion layer sitting in front of Spark Streaming in real production architectures, and this unit treats it with the same depth as any Spark topic — its own cluster architecture, its own installation process, and its own delivery guarantees — before connecting it back to Spark.

Following Units III and IV's lead, expect dense, runnable code throughout: Kafka CLI commands, Scala producer/consumer code, and Spark Structured Streaming code reading directly from Kafka topics, plus seven new diagrams covering Kafka's internals, cluster architecture, the producer/consumer model, Spark integration, ingestion pipelines, and delivery semantics.



The Postal Service Standing Between Every Sender and Every Reader

Every prior unit assumed data was already sitting somewhere waiting to be read -- a file in HDFS, a table in Spark SQL. Kafka is what exists BEFORE that: a tireless, durable postal service standing between every application that produces events (a website logging clicks, a payment system logging transactions) and every application that wants to consume them (Spark, a monitoring dashboard, a fraud detector) -- accepting mail continuously, storing it reliably even if the recipient hasn't checked their mailbox yet, and letting multiple completely different recipients each read the SAME mail independently, at their own pace, without one recipient's mailbox habits affecting anyone else's.

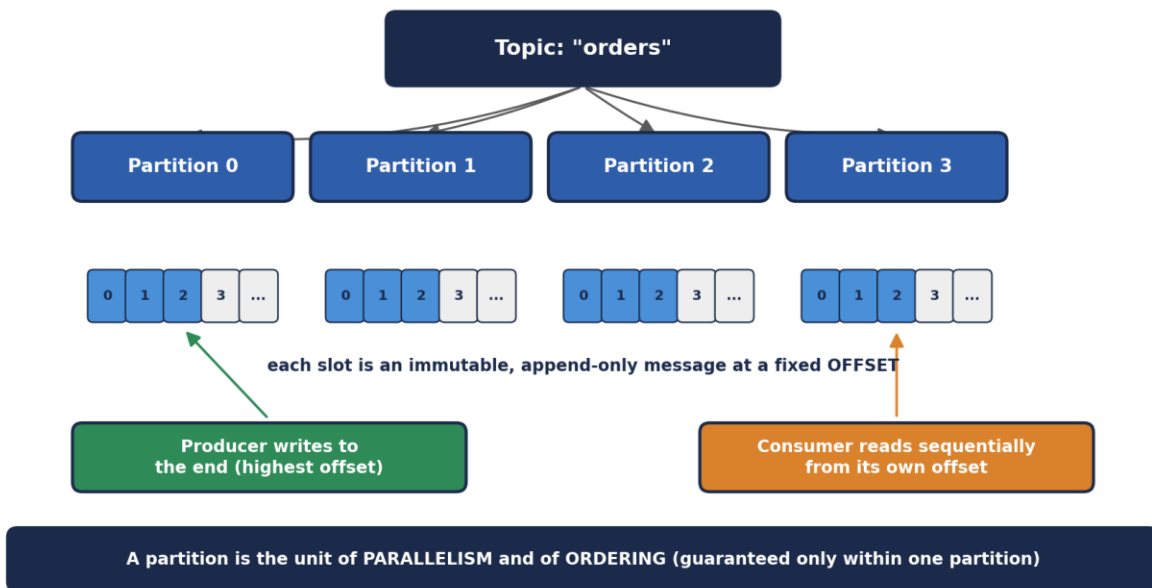
1. Fundamentals of Apache Kafka

Apache Kafka is a distributed event streaming platform, originally built at LinkedIn and now an Apache project, designed to handle high-throughput, real-time data feeds reliably. This section builds Kafka's core vocabulary from the ground up.

1.1 Events, Topics, and Partitions

An EVENT (also called a message or record) is the basic unit of data in Kafka -- a single fact that happened, like "user 42 clicked button X at time T." Events are organized into TOPICS, a named, logical channel ("clickstream", "orders", "sensor-readings"). Every topic is split into one or more PARTITIONS, and this is the single most important structural fact about Kafka.

Kafka Fundamentals — Topics, Partitions, and Offsets



Kafka Fundamentals — Topics, Partitions, and Offsets

- Partition — an ordered, immutable, append-only log. New events are always appended to the end; existing events are never modified or deleted (until a retention policy expires them).
- Offset — a sequential ID number identifying an event's exact position within its partition. Offsets are only meaningful WITHIN one partition, not across the whole topic.
- Ordering guarantee — Kafka guarantees event order ONLY within a single partition, never across partitions of the same topic. This is a deliberate trade-off enabling parallelism (Section 5), and a very common interview question.

```
# Creating a topic with 4 partitions and replication factor 3
kafka-topics.sh --create \
  --bootstrap-server localhost:9092 \
  --topic clickstream \
  --partitions 4 \
  --replication-factor 3

# List all topics
kafka-topics.sh --list --bootstrap-server localhost:9092

# Describe a topic -- see its partitions, leaders, and replicas
kafka-topics.sh --describe --bootstrap-server localhost:9092 --topic clickstream
```

1.2 Brokers, Retention, and Durability

A **BROKER** is a single Kafka server process; a **Kafka CLUSTER** is one or more brokers working together (Section 2 covers this in depth). Every message written to Kafka is durably persisted to disk before being acknowledged -- Kafka is not an in-memory queue, and does not lose data when a consumer is slow or offline. Messages are kept according to a configurable **RETENTION** policy (commonly time-based, e.g. 7 days) rather than being deleted the moment they're read, which is precisely what allows multiple independent consumers to each read the same topic at their own pace.

```
# Set a topic's retention to 3 days (259200000 ms)
kafka-configs.sh --bootstrap-server localhost:9092 \
  --entity-type topics --entity-name clickstream \
  --alter --add-config retention.ms=259200000
```

1.3 Why Kafka, Specifically, for Streaming Data

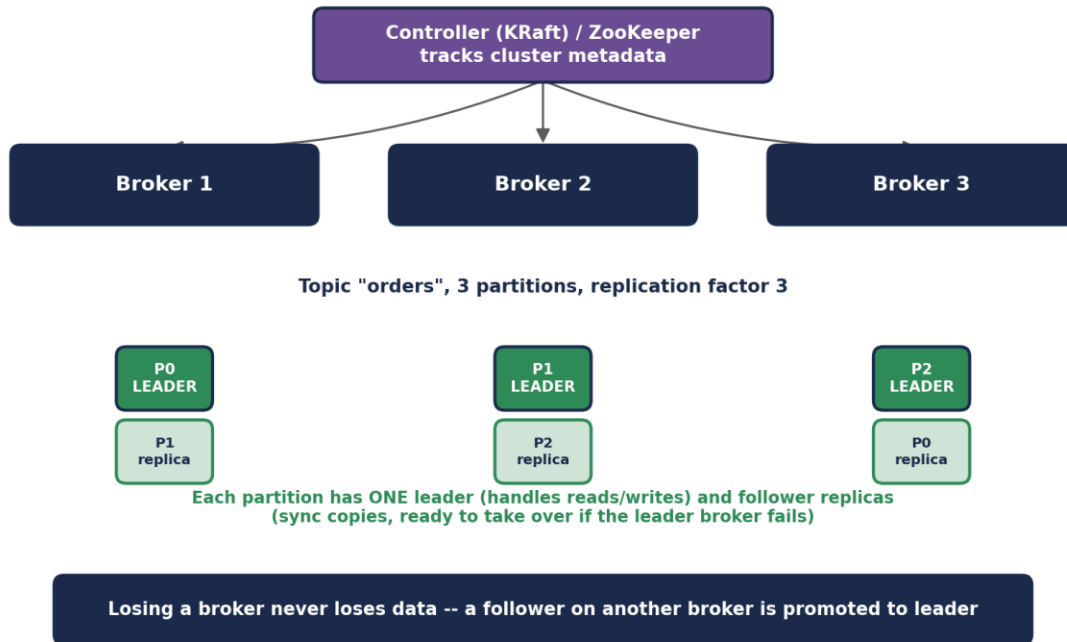
Property	Why It Matters
High throughput	Designed to handle millions of events/second across a cluster
Durability	Every message is written to disk and replicated before being acknowledged
Decoupling	Producers and consumers never talk to each other directly — only to Kafka
Replayability	Consumers can re-read historical data within the retention window, not just "new" events
Horizontal scalability	Add more brokers and partitions to handle more load, the same principle as Spark's own scaling

Property	Why It Matters
 Kafka's role in the modern data stack Kafka has become the de facto standard "nervous system" connecting operational systems (a website, a payment processor) to analytical systems (Spark, a data warehouse) in real-time architectures across finance, e-commerce, and telecom. Its specific value proposition — durable, replayable, decoupled event delivery at very high throughput — is precisely why it, rather than a simpler message queue, sits at the center of most Lambda/Kappa-style architectures from Unit I.	
 A Library With an Infinite, Ever-Growing Scroll <i>Picture each Kafka partition as a single, continuously-growing paper scroll in a library. New entries are only ever added to the RIGHT END of the scroll -- nobody erases or edits an entry already written. A reader (consumer) can start reading from any position on the scroll they like -- the very beginning, somewhere in the middle, or right where they left off last time -- and their reading pace never affects any other reader's own bookmark on that same scroll. The library simply keeps stocking new scroll material at the end, and keeps old material around for a while (retention) in case anyone still needs to look back.</i>	
 Check yourself — Section 1 Q1. Why does Kafka guarantee ordering only within a partition, not across an entire topic? Q2. What does Kafka's retention policy allow that a simple "delete after read" message queue would not? Q3. Why is Kafka described as "decoupling" producers from consumers? Hands-on Challenge: Using a local Kafka installation (Section 3 covers setup), create a topic with 3 partitions and a 1-day retention policy, then use <code>`kafka-topics.sh --describe`</code> to confirm the partition count and leader assignment.	

2. Apache Kafka Cluster Architecture

This section zooms out from a single topic to the full cluster: how multiple brokers coordinate, how replication provides fault tolerance, and how cluster metadata is managed.

Apache Kafka Cluster Architecture — Brokers & Replication



Apache Kafka Cluster Architecture — Brokers & Replication

2.1 Brokers and the Controller

Each BROKER in a cluster stores a subset of the cluster's partitions and serves read/write requests for them. One broker is elected CONTROLLER, responsible for cluster-wide administrative decisions: which broker leads which partition, detecting broker failures, and triggering leader re-election when needed. Historically this coordination relied on an external Apache ZooKeeper ensemble; modern Kafka versions (2.8+) can run this metadata layer internally via KRaft (Kafka Raft), removing the ZooKeeper dependency entirely.

2.2 Replication — Leaders and Followers

Every partition has a configurable REPLICATION FACTOR (commonly 3 in production). One replica is elected LEADER and handles all reads and writes for that partition; the remaining replicas are FOLLOWERS, which continuously copy the leader's data. If the broker hosting the leader fails, one of the in-sync followers is automatically promoted to leader -- directly analogous to HDFS's block replication and Kafka's own inspiration, Google's distributed systems literature, from earlier units.


```
# Describe a topic to see exactly which broker leads each partition,
# and which brokers hold in-sync replicas (ISR)
kafka-topics.sh --describe --bootstrap-server localhost:9092 --topic orders

# Example output:
# Topic: orders Partition: 0 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3
# Topic: orders Partition: 1 Leader: 2 Replicas: 2,3,1 Isr: 2,3,1
# Topic: orders Partition: 2 Leader: 3 Replicas: 3,1,2 Isr: 3,1,2
```

ISR — In-Sync Replicas — is the detail that actually matters

A follower only counts as truly "safe" if it's in the ISR (In-Sync Replica) set — meaning it has genuinely caught up to the leader's recent writes, not just nominally listed as a replica. A follower that has fallen too far behind (a slow or partially failed broker) is dropped from the ISR and cannot be promoted to leader until it catches back up, precisely to avoid promoting a replica that would silently lose recent, unreplicated data.

2.3 Consumer Groups and Cluster-Wide Coordination

Beyond storage, the cluster also coordinates CONSUMER GROUPS (fully explored in Section 5) — tracking which consumer in a group is responsible for which partition, and reassigning partitions automatically if a consumer joins or leaves the group.

A Chain of Regional Warehouses With Backup Managers

Think of a Kafka cluster as a chain of regional warehouses (brokers), where each warehouse's inventory (a partition) has one officially responsible MANAGER (the leader) and a couple of trained ASSISTANT MANAGERS (followers) at OTHER warehouse locations who are kept fully briefed on every single inventory change, in real time. If the manager's warehouse burns down, one of the fully-briefed assistant managers at a different location instantly steps up and takes over -- because they already knew everything the original manager knew, having been kept perfectly in sync the whole time.

Why replication factor is a real cost/reliability trade-off

Replication factor 3 (the common production default) means every byte of data is stored on three different brokers, tripling the storage cost compared to no replication — a deliberate, explicit trade-off between durability and infrastructure spend, tuned per-topic based on how critical that data actually is. A topic holding easily-regenerable, low-value debug logs might run at replication factor 1 or 2, while a topic holding financial transactions almost always runs at 3 or higher.

Check yourself — Section 2

Q1. What specific job does the Controller do that an ordinary broker does not?

Q2. Why isn't every replica of a partition automatically eligible to become the new leader if the current leader fails?

Q3. What real cost does a higher replication factor impose, and why might a team choose a lower one for some topics?

Hands-on Challenge:

If you have a local Kafka cluster (even a single-broker one for testing), run ``kafka-topics.sh --describe`` on any topic and identify the Leader, Replicas, and Isr fields in the output. Explain in your own words what would happen if the leader broker were stopped.

3. Apache Kafka Installation

This section walks through a complete local Kafka installation, directly mirroring the hands-on installation approach from Unit I's Spark setup -- download, extract, configure, start the daemons, and verify.

3.1 Prerequisites

- A working JDK, exactly as required for Spark (Kafka is also a JVM application, written primarily in Scala and Java).
- Enough free disk space — Kafka persists every message to disk by design (Section 1.2).
- A recent Kafka release, which since version 2.8+ can run WITHOUT a separate ZooKeeper installation using KRaft mode — the approach shown below.

3.2 Downloading and Extracting Kafka

```
# 1. Download a Kafka release
cd ~
wget https://downloads.apache.org/kafka/3.7.0/kafka_2.13-3.7.0.tgz

# 2. Extract it
tar -xvzf kafka_2.13-3.7.0.tgz
sudo mv kafka_2.13-3.7.0 /opt/kafka
sudo chown -R $(whoami):$(whoami) /opt/kafka

# 3. Add Kafka's bin/ directory to PATH (same pattern as Unit I's Spark setup)
echo 'export PATH=$PATH:/opt/kafka/bin' >> ~/.bashrc
source ~/.bashrc
```

3.3 Starting Kafka in KRaft Mode (No ZooKeeper Required)

```
# 1. Generate a unique cluster ID (only needs to be done ONCE)
KAFKA_CLUSTER_ID=$(kafka-storage.sh random-uuid)
echo $KAFKA_CLUSTER_ID

# 2. Format the storage directory with that cluster ID
kafka-storage.sh format \
  -t $KAFKA_CLUSTER_ID \
  -c /opt/kafka/config/kraft/server.properties

# 3. Start the Kafka broker (runs in the foreground -- use & or a separate terminal)
kafka-server-start.sh /opt/kafka/config/kraft/server.properties
```

```
# You should see log lines ending with something like:
# [KafkaServer id=1] started (kafka.server.KafkaServer)
```

✓ The classic ZooKeeper-based startup, still common in older deployments

Many existing production clusters and course materials still use the older two-daemon startup: `zookeeper-server-start.sh config/zookeeper.properties` first, then `kafka-server-start.sh config/server.properties` pointing at that ZooKeeper. Both approaches produce a working cluster; KRaft mode is the direction the project has moved toward, but recognizing the older ZooKeeper-based flow is still useful for working with existing infrastructure.

3.4 Creating a Topic and Verifying the Installation

```
# Create a test topic
kafka-topics.sh --create --topic test-topic \
  --bootstrap-server localhost:9092 \
  --partitions 3 --replication-factor 1

# Confirm it exists
kafka-topics.sh --list --bootstrap-server localhost:9092

# Send a few test messages from the command line
kafka-console-producer.sh --topic test-topic --bootstrap-server localhost:9092
> hello kafka
> this is a test message
# (Ctrl+D to exit)

# Read them back, from the beginning, in a SEPARATE terminal
kafka-console-consumer.sh --topic test-topic \
  --bootstrap-server localhost:9092 --from-beginning
# hello kafka
# this is a test message
```

⚠ A very common local setup mistake

Running Kafka locally with `--replication-factor` greater than 1 on a SINGLE broker will fail with an error, since replication factor cannot exceed the number of available brokers — for single-broker local testing, always use `--replication-factor 1`, and only test higher replication factors once multiple broker instances are actually running.

📦 Setting Up a New Post Office Branch

Installing Kafka is opening a brand-new post office branch: unpack the building materials (extract the tarball), register the branch with the regional headquarters (format storage with a cluster ID), open the

doors for business (start the broker), and then do a test run -- mail a letter to yourself (produce a message) and check that it actually arrives in the mailbox (consume it back) before trusting the branch with real customer mail.

Why local installation skills transfer directly to production ops

The exact same commands used here for local testing — ``kafka-topics.sh``, ``kafka-console-producer.sh``, ``kafka-console-consumer.sh`` — are the same tools operations engineers reach for daily when diagnosing a production Kafka cluster: checking whether a topic exists, manually producing a test message to confirm connectivity, or tailing a topic's live traffic during an incident. There is no separate "production toolkit" to learn later — this IS the toolkit.

Check yourself — Section 3

- Q1. Why does modern Kafka not require a separate ZooKeeper installation, and what replaced it?
- Q2. What would happen if you tried to create a topic with ``--replication-factor 3`` against a single-broker cluster?
- Q3. What are the two console tools used to manually produce and consume test messages, and why are they useful beyond just learning?

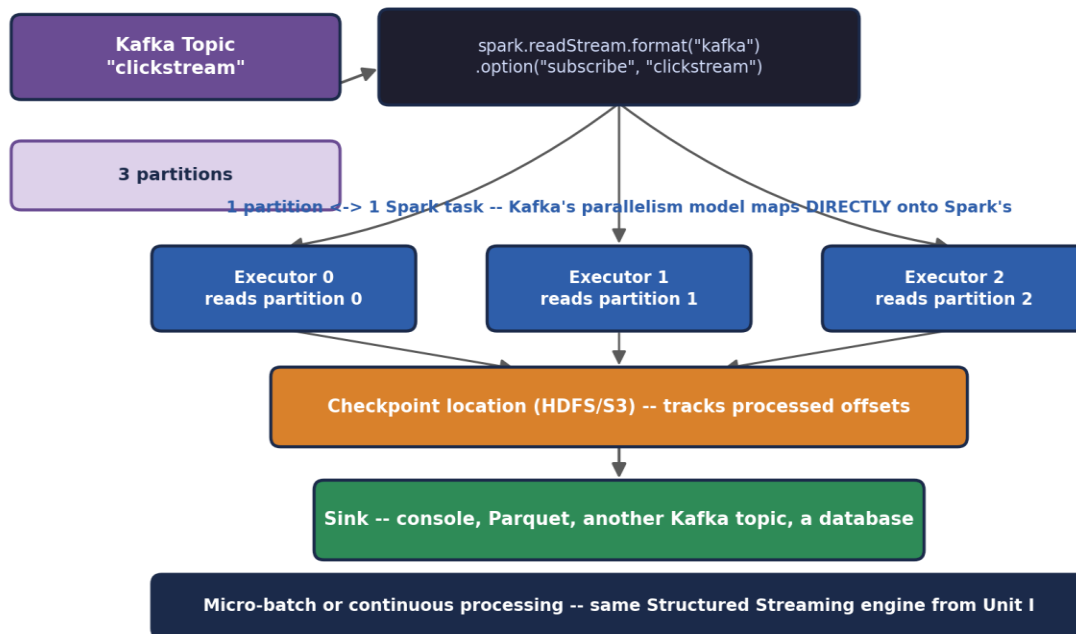
Hands-on Challenge:

Complete the full installation above end to end on your own machine (or a free cloud shell/VM): download and extract Kafka, format storage in KRaft mode, start the broker, create a topic with 3 partitions, and produce/consume at least 3 test messages using the console tools.

4. Integration of Apache Kafka with Spark

This section connects Sections 1-3's Kafka foundation directly to Spark Structured Streaming, first introduced conceptually in Unit I. The integration is remarkably natural precisely because Kafka's partitioning model and Spark's task-parallelism model were designed around the same core idea.

Integrating Apache Kafka With Spark Structured Streaming



Integrating Apache Kafka With Spark Structured Streaming

4.1 Reading From Kafka as a Streaming Source

```
import org.apache.spark.sql.SparkSession

val spark = SparkSession.builder().appName("KafkaStreamDemo").getOrCreate()

val kafkaStream = spark.readStream
  .format("kafka")
  .option("kafka.bootstrap.servers", "localhost:9092")
  .option("subscribe", "clickstream")
  .option("startingOffsets", "earliest") // or "latest" for only new data
  .load()

// Kafka messages arrive with a FIXED schema: key, value, topic, partition,
// offset, timestamp, timestampType -- all as raw bytes
```

```
kafkaStream.printSchema()
// root
// |-- key: binary
// |-- value: binary
// |-- topic: string
// |-- partition: integer
// |-- offset: long
// |-- timestamp: timestamp
// |-- timestampType: integer
```

4.2 Parsing Kafka's Raw Bytes Into Structured Data

Kafka has no concept of your application's data shape -- it deals purely in bytes. The value column (and often key) must be explicitly cast and parsed, commonly from JSON, directly reusing Unit IV's DataFrame schema tools.

```
import org.apache.spark.sql.functions._
import org.apache.spark.sql.types._

val eventSchema = StructType(Array(
  StructField("userId", StringType),
  StructField("action", StringType),
  StructField("timestamp", LongType)
))

val parsed = kafkaStream
  .selectExpr("CAST(value AS STRING) as json_str")
  .select(from_json($"json_str", eventSchema).as("data"))
  .select("data.*")

parsed.printSchema()
// root
// |-- userId: string
// |-- action: string
// |-- timestamp: long
```

4.3 Processing and Writing Results

Once parsed, a streaming DataFrame supports the EXACT same operations as Unit IV's batch DataFrames -- groupBy, agg, filter, join -- Spark's engine handles translating them into an incrementally-updated streaming computation automatically.

```
val actionCounts = parsed
  .groupBy("action")
  .count()
```

```

val query = actionCounts.writeStream
  .outputMode("complete") // "append", "update", or "complete"
  .format("console")
  .option("checkpointLocation", "hdfs:///checkpoints/click-counts")
  .start()

query.awaitTermination()

// Writing the RESULTS of a streaming computation back to ANOTHER Kafka topic
// -- a common pipeline pattern (raw events in, enriched events out)
val enriched = parsed.withColumn("processedAt", current_timestamp())
enriched.selectExpr("CAST(userId AS STRING) AS key", "to_json(struct(*)) AS
value")
  .writeStream
  .format("kafka")
  .option("kafka.bootstrap.servers", "localhost:9092")
  .option("topic", "enriched-events")
  .option("checkpointLocation", "hdfs:///checkpoints/enriched")
  .start()

```

4.4 Delivery Semantics — What Actually Happens on Failure

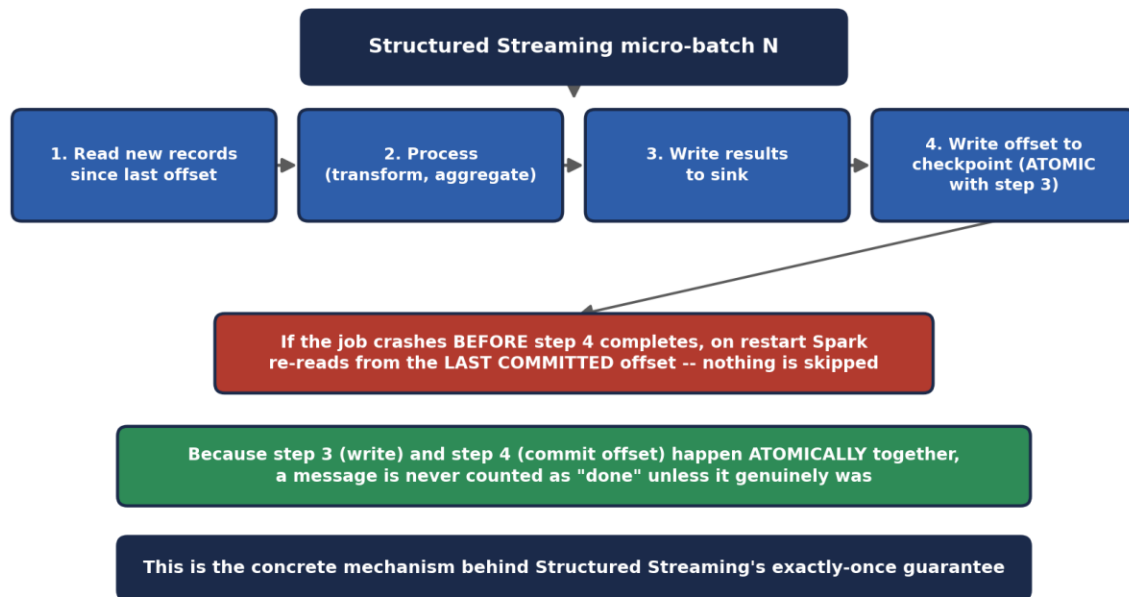
Message Delivery Semantics

At-Most-Once	At-Least-Once	Exactly-Once
Commit offset BEFORE processing	Process message BEFORE committing	Process + commit offset in ONE atomic transaction
If a crash happens mid-process...	If a crash happens after processing...	Spark checkpointing + idempotent sinks
...that message is LOST, never retried	...message is REPROCESSED (duplicate possible)	Every message affects the result exactly once

Spark's default Kafka integration achieves EXACTLY-ONCE via checkpointed offsets + idempotent writes

Message Delivery Semantics

Offset Tracking & Checkpointing — How Exactly-Once Actually Works



Offset Tracking & Checkpointing — How Exactly-Once Actually Works

Spark's Kafka integration achieves EXACTLY-ONCE processing semantics by combining Kafka's own offset tracking with Spark's checkpointing (the `checkpointLocation` option above): the offset marking "how far we've read" is only advanced ATOMICALLY together with the corresponding output being durably written, so a mid-batch failure and restart replays exactly the unprocessed range -- never less, never duplicated in the final result (assuming an idempotent sink).

⚠ The sink must cooperate for true exactly-once

Spark's checkpointing guarantees exactly-once READING and PROCESSING, but the final END-TO-END guarantee also depends on the SINK. Writing to an idempotent sink (one where writing the same result twice has the same effect as writing it once, like an upsert into a database keyed by a unique ID) preserves exactly-once semantics. A sink like a plain append-only file or an email-sending action is NOT naturally idempotent, and can still produce duplicate side effects on a retry even with perfect Spark checkpointing.



A Translator Standing Between Two People Who Don't Speak the Same Language

Kafka speaks in raw, opaque bytes -- to Kafka, a message is just a blob, no different from any other blob. Spark's DataFrame world speaks in named, typed columns. The `'from_json' + schema` pattern in Section 4.2 is a translator standing at the border between these two worlds, taking Kafka's opaque bytes and turning them into something Spark's DataFrame engine -- and everything you learned in Unit IV -- can actually reason about, filter, join, and aggregate.

Why this specific combination dominates real-time architectures

Kafka-plus-Spark-Structured-Streaming is, by a wide margin, the most common real-time processing stack in production Big Data systems today, precisely because Kafka's durability and replayability (Section 1) pair naturally with Spark's exactly-once checkpointing and its shared DataFrame API with batch processing (Unit IV) — a single team can write nearly identical code for both their nightly batch reports and their real-time dashboards.

Check yourself — Section 4

Q1. What columns does a raw Kafka streaming DataFrame arrive with BEFORE any parsing, and why is `value` typed as binary rather than string?

Q2. What two things must happen ATOMICALLY together for Spark's Kafka integration to achieve exactly-once processing?

Q3. Why does true end-to-end exactly-once delivery also depend on the SINK being idempotent, not just on Spark's checkpointing?

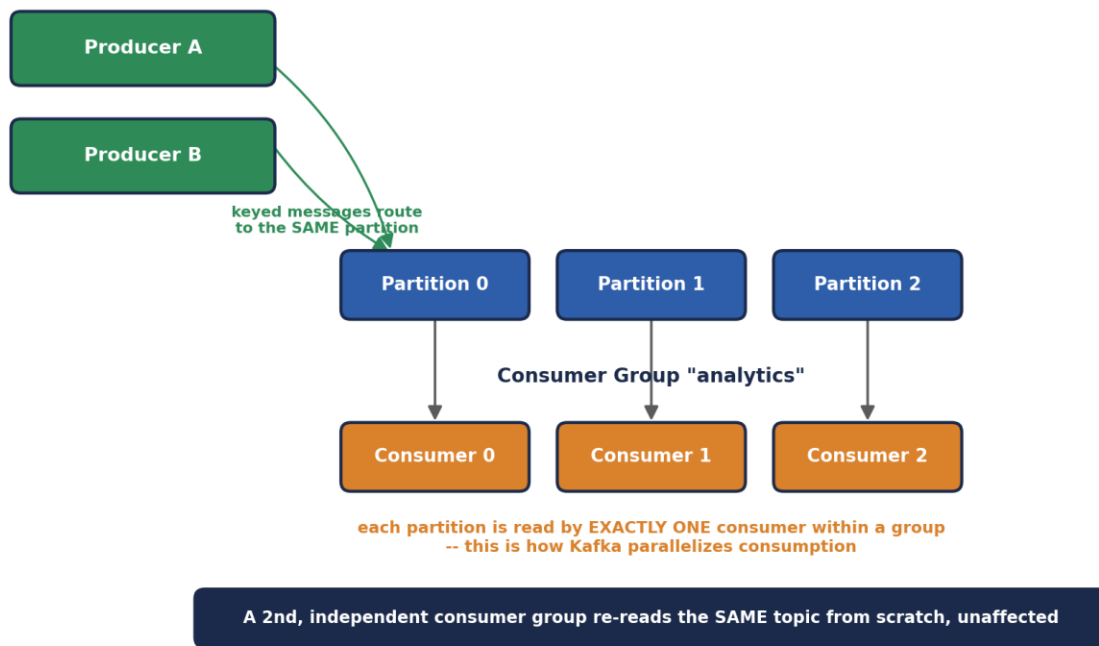
Hands-on Challenge:

Set up a local Kafka topic (Section 3), write a small Spark Structured Streaming job that reads from it, parses a simple JSON schema of your choosing, computes a running count grouped by one field, and writes the result to the console. Produce a few test messages and confirm the counts update as expected.

5. Producer & Consumer Messaging Model

This section goes deeper into the two client roles briefly mentioned throughout Sections 1-4: PRODUCERS, which write events into Kafka, and CONSUMERS, which read them -- including the CONSUMER GROUP mechanism that is Kafka's core parallelism model.

Producer & Consumer Messaging Model



Producer & Consumer Messaging Model

5.1 Producers — Writing Events

A producer sends key-value messages to a topic. If a KEY is provided, Kafka hashes it to deterministically choose a partition -- guaranteeing every message with the SAME key always lands in the SAME partition, which is precisely what preserves per-key ordering (Section 1.1) even though ordering across the whole topic isn't guaranteed.

```
import org.apache.kafka.clients.producer._
import java.util.Properties

val props = new Properties()
props.put("bootstrap.servers", "localhost:9092")
props.put("key.serializer",
  "org.apache.kafka.common.serialization.StringSerializer")
```

```

props.put("value.serializer",
"org.apache.kafka.common.serialization.StringSerializer")

val producer = new KafkaProducer[String, String](props)

// Messages with the SAME key ('user42') always go to the SAME partition,
// preserving order for that specific user's events
val record = new ProducerRecord[String, String]("clickstream", "user42",
""""{"action":"click"}""")
producer.send(record)

// send() is ASYNCHRONOUS by default -- attach a callback to confirm delivery
producer.send(record, (metadata: RecordMetadata, exception: Exception) => {
  if (exception != null) println(s"Send failed: ${exception.getMessage}")
  else println(s"Sent to partition ${metadata.partition()} at offset
${metadata.offset()}")
})

producer.flush()
producer.close()

```

5.2 Producer Acknowledgment Levels

acks setting	Guarantee
acks=0	Fire and forget — fastest, but messages can be silently lost
acks=1	Leader confirms the write — safe unless the leader fails before followers replicate
acks=all (-1)	ALL in-sync replicas confirm — strongest durability, used for critical data

5.3 Consumers and Consumer Groups

A consumer group is a set of consumers cooperatively reading a topic, with Kafka automatically assigning each partition to exactly ONE consumer within the group at a time — this is the exact mechanism that lets you scale consumption horizontally, directly parallel to Spark's own partition-to-task assignment from Unit III.

```

import org.apache.kafka.clients.consumer._
import java.util.{Collections, Properties}

val props = new Properties()
props.put("bootstrap.servers", "localhost:9092")
props.put("group.id", "analytics-group")

```

```

props.put("key.deserializer",
"org.apache.kafka.common.serialization.StringDeserializer")
props.put("value.deserializer",
"org.apache.kafka.common.serialization.StringDeserializer")
props.put("auto.offset.reset", "earliest") // where to start if no offset is
committed yet

val consumer = new KafkaConsumer[String, String](props)
consumer.subscribe(Collections.singletonList("clickstream"))

while (true) {
  val records = consumer.poll(java.time.Duration.ofMillis(1000))
  records.forEach { record =>
    println(s"partition=${record.partition()} offset=${record.offset()}
value=${record.value()}")
  }
  consumer.commitSync() // manually commit offsets AFTER successfully
processing
}

```

A key rule for scaling consumers

Adding MORE consumers to a group than a topic has partitions doesn't help — the extra consumers sit idle, since a partition can only be assigned to one consumer within a group at a time. If a topic has 4 partitions, the maximum useful parallelism within ONE consumer group is 4 consumers; a 5th would have nothing to read. This is a direct, common capacity-planning question in production Kafka deployments.

5.4 Committing Offsets — Manual vs. Automatic

```

// Automatic offset commits (simpler, but risks the at-least/at-most-once
// trade-offs from Section 4.4 depending on WHEN the commit happens relative
// to processing)
props.put("enable.auto.commit", "true")
props.put("auto.commit.interval.ms", "5000")

// Manual commits -- full control over EXACTLY when an offset counts as "done",
// which is what lets an application implement its own at-least-once or
// exactly-once handling on top of the raw consumer API
props.put("enable.auto.commit", "false")
// ... after successfully processing a batch of records:
consumer.commitSync()

```

A Kitchen With Exactly One Cook Per Station

A consumer group is a restaurant kitchen with exactly as many prep stations (partitions) as there are dedicated cooks needed -- assign one cook per station, and the kitchen runs at full efficiency. Hire a 5th

cook for a 4-station kitchen, and that cook just stands around with nothing to chop -- there's no station left to give them. If a cook calls in sick (a consumer crashes or leaves the group), the head chef (Kafka's group coordinator) immediately reassigns their station to one of the remaining cooks, so nothing goes unprepped.

Why the producer/consumer split matters architecturally

Because producers and consumers never communicate directly — only through Kafka — a producing application (a web server) can be scaled, restarted, or entirely rewritten without the consuming application (a Spark job) ever noticing, and vice versa. This decoupling is precisely the property that lets large organizations run dozens of independently-developed, independently-deployed services all producing and consuming from a shared set of Kafka topics without tight coordination between every team.

Check yourself — Section 5

Q1. Why do messages with the same key always land in the same partition, and why does that matter for ordering?

Q2. What happens if you add a 5th consumer to a group reading a 4-partition topic?

Q3. What's the practical difference between `acks=1` and `acks=all`, and when would you choose the stronger guarantee?

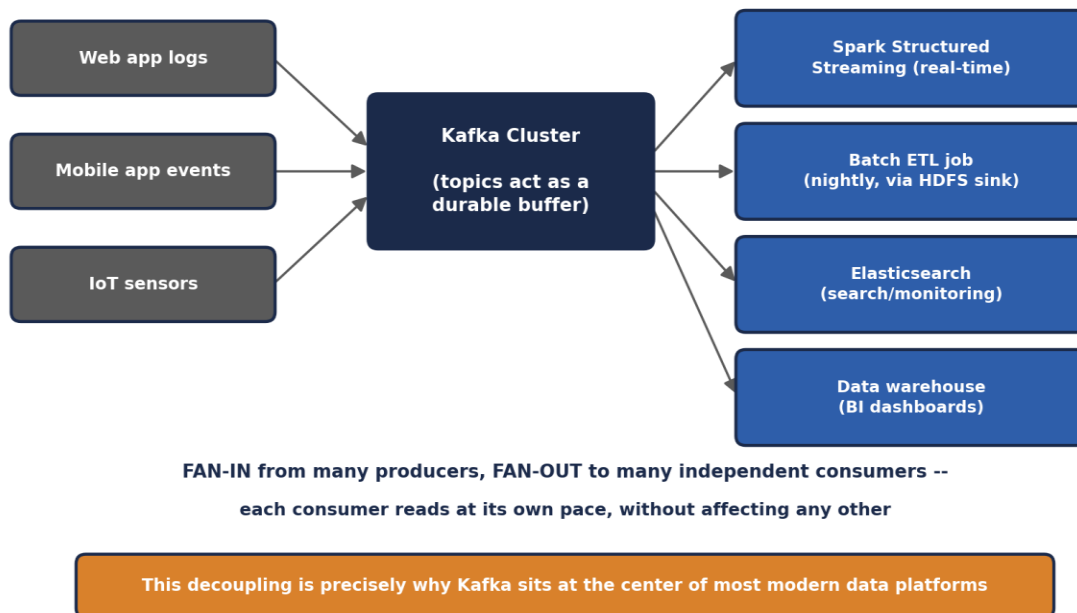
Hands-on Challenge:

Write a small Scala (or Python) Kafka producer that sends 20 keyed messages to a topic with 3 partitions, using only 4 distinct keys. Then write a consumer in a group and observe, using the partition field in each record, that all messages sharing a key consistently land in the same partition.

6. Kafka Pipeline

This section zooms out from individual producers and consumers (Section 5) to how they combine into full, end-to-end PIPELINES -- the shape Kafka takes in a real data platform, connecting many sources to many destinations through one durable, decoupled hub.

A Typical Kafka Ingestion Pipeline



A Typical Kafka Ingestion Pipeline

6.1 Fan-In: Many Producers, One Topic

A single topic commonly receives events from MANY independent producers simultaneously -- every web server in a load-balanced fleet, every microservice instance, every mobile device. Kafka handles this natively: any number of producers can write to the same topic concurrently, and the cluster (Section 2) simply distributes the resulting load across the topic's partitions.

```

# Simulating fan-in: multiple producer processes writing to the same topic
# Producer instance 1 (e.g. running on web-server-01)
kafka-console-producer.sh --topic clickstream --bootstrap-server localhost:9092

# Producer instance 2 (e.g. running on web-server-02), completely independent
kafka-console-producer.sh --topic clickstream --bootstrap-server localhost:9092

# Both write into the SAME topic; Kafka doesn't distinguish which
  
```

```
# physical machine or process a message came from unless the application
# explicitly includes that information in the message itself
```

6.2 Fan-Out: One Topic, Many Independent Consumers

Recall Section 1.2's retention model: because messages aren't deleted on read, the SAME topic can simultaneously feed a real-time Spark Structured Streaming job, a separate batch ETL process reading periodically, a search-indexing consumer, and a monitoring dashboard -- each running as its OWN consumer group (Section 5.3), completely unaware of and unaffected by the others' pace or even existence.

```
// Spark Structured Streaming consumer group (implicit, managed by Spark)
spark.readStream.format("kafka")
  .option("subscribe", "clickstream")
  .option("kafka.group.id", "spark-realtime-analytics")
  .load()

# A completely separate consumer group, reading the SAME topic
# for an entirely different purpose (e.g. archiving to cold storage)
kafka-console-consumer.sh --topic clickstream \
  --bootstrap-server localhost:9092 \
  --group archival-service --from-beginning
```

6.3 A Complete Pipeline, End to End

Pulling Sections 1-6 together, a realistic pipeline looks like this: application servers (producers) send click events into a clickstream topic; Kafka durably stores and replicates them across the cluster (Section 2); a Spark Structured Streaming job (Section 4) reads the topic as one consumer group, computing real-time aggregates written to a dashboard; simultaneously, a separate nightly Spark batch job reads the SAME topic (a different consumer group, or even reading via Spark SQL against a Kafka-backed sink) to build the complete historical record for reporting -- exactly the Lambda-style pattern from Unit I, but with Kafka as the shared, durable backbone connecting both paths instead of two entirely separate ingestion systems.



A Central Train Station, Not a Point-to-Point Taxi Fleet

Without Kafka, connecting 5 event-producing services directly to 4 event-consuming systems means building and maintaining potentially 20 separate point-to-point connections -- a taxi fleet where every rider has to personally arrange a private car to every possible destination. Kafka is a central train station: every producer just needs to know how to get to the station (one topic), and every consumer just needs to know which platform to wait on (subscribe to that topic) -- 5 plus 4 connections total, not 20, and adding a 6th producer or a 5th consumer means connecting just that ONE new party to the station, not renegotiating every other connection in the system.

Why this fan-in/fan-out shape is the actual reason companies adopt Kafka

The single most cited reason large organizations adopt Kafka isn't raw throughput — it's this decoupling. Before Kafka-style architectures, adding a new data consumer often meant negotiating a new direct integration with every relevant producing system. With Kafka already sitting in the middle, a new consuming system (a new ML model, a new dashboard, a new compliance-archiving process) can simply subscribe to already-existing topics with zero changes required to any producer.

Check yourself — Section 6

Q1. Why can multiple independent producers write to the same Kafka topic without any special coordination between them?

Q2. What specifically allows a real-time Spark job AND a nightly batch job to both consume from the same Kafka topic without interfering with each other?

Q3. In the train-station analogy, what does adding a new consumer cost in terms of changes to existing producers?

Hands-on Challenge:

Sketch (on paper) a pipeline of your own design with at least 3 distinct producers feeding one topic, and at least 3 distinct, independent consumer groups reading from it for different purposes. Label what each producer and consumer represents.

7. Ingestion Patterns

This closing section catalogs the common, named PATTERNS for getting data into Kafka in the first place -- the specific architectural choices real teams make when connecting existing systems (databases, applications, files) to a Kafka-based pipeline.

7.1 Direct Application Instrumentation

The most straightforward pattern: an application directly produces events to Kafka as part of its own normal code, exactly as shown in Section 5.1's producer example -- a web server emits a click event, a mobile app emits a usage event, directly and deliberately, as first-class application behavior.

Pattern	When It's the Right Choice
Direct instrumentation	You control the source application's code and want fine-grained, deliberate event emission
Change Data Capture (CDC)	Data already lives in a database and you need its CHANGES streamed without modifying the source app
Log-based ingestion	Existing systems already write structured logs, and rewriting them to produce directly isn't practical
Batch-to-stream bridging	An existing batch system's periodic output needs to feed into a streaming pipeline

7.2 Change Data Capture (CDC)

Rather than modifying an existing application to explicitly produce Kafka events, CDC tools (Debezium is the most common open-source example) tail a database's own internal transaction/replication log and automatically publish every INSERT, UPDATE, and DELETE as a Kafka event -- capturing changes with zero modification to the original application's code.

```
// A simplified illustration of the SHAPE of a CDC event Debezium produces
// for an UPDATE on an 'orders' table -- note it captures BOTH the before
// and after state of the row
{
  "before": { "order_id": 1042, "status": "pending", "amount": 250.0 },
  "after": { "order_id": 1042, "status": "shipped", "amount": 250.0 },
  "source": { "table": "orders", "ts_ms": 1719580200000 },
  "op": "u"    // c = create, u = update, d = delete, r = read (initial
              // snapshot)
}
```

Spark Structured Streaming can consume CDC events from Kafka exactly like any other JSON payload (Section 4.2), making CDC-based ingestion a common way to build a near-real-time replica of an operational database inside a Big Data platform without touching the original application at all.

7.3 Batch-to-Stream Bridging

Not every data source is naturally a stream. A nightly export from a legacy mainframe or a periodic file drop from a partner organization is inherently BATCH-shaped, yet still needs to feed a Kafka-based pipeline. A common bridging pattern reads each new batch file as it arrives and produces its rows as individual Kafka events -- effectively converting a batch source into a stream source at the ingestion boundary, so everything downstream of Kafka can remain uniformly stream-oriented.

```
// Reading a batch file with Spark and producing each row as a Kafka event --
// bridging Unit IV's batch DataFrame world into a Kafka stream
val batchDF = spark.read.option("header",
"true").csv("hdfs:///drops/daily_export.csv")

batchDF
  .selectExpr("CAST(id AS STRING) AS key", "to_json(struct(*)) AS value")
  .write
  .format("kafka")
  .option("kafka.bootstrap.servers", "localhost:9092")
  .option("topic", "daily-export-events")
  .save()
```

7.4 Choosing an Ingestion Pattern



Four Different Ways to Get Mail Into the Postal System

Direct instrumentation is writing and mailing your own letter yourself, deliberately, whenever you have something to say. CDC is a silent scribe sitting behind a busy office worker, invisibly copying every memo the moment it's filed, without the worker ever being asked to do anything differently. Log-based ingestion is someone finding an old diary that was already being kept for other reasons and deciding to mail out photocopies of each new page as it's written. Batch-to-stream bridging is taking a single huge annual report that arrives once a year and, at the post office counter, splitting it apart and mailing out its individual paragraphs as separate letters, so everything downstream only ever has to deal with individual letters, never a giant annual bundle.



Why CDC in particular has become so widely adopted

CDC's specific appeal is that it requires ZERO changes to existing, often business-critical applications that teams are understandably reluctant to modify — a legacy order-management system nobody wants to risk touching can still feed a real-time analytics pipeline via CDC, capturing every change at the database

level without a single line of that legacy application's code being altered. This makes CDC a common first step when organizations want real-time capabilities without a risky rewrite of core systems.

Check yourself — Section 7

Q1. What's the key advantage of Change Data Capture over modifying an application to directly produce Kafka events?

Q2. In a CDC event's JSON payload, what's the difference between the `before` and `after` fields, and when would `before` be null?

Q3. Why might a team choose batch-to-stream bridging instead of just leaving a periodic file-drop process as pure batch?

Hands-on Challenge:

Design (on paper) an ingestion strategy for a hypothetical retail company with three data sources: a legacy order-management database nobody wants to modify, a modern mobile app your team controls, and a partner organization that drops a CSV file once daily. Choose and justify one ingestion pattern for each.

Unit V Review

Quick Review Quiz

Three questions covering the full breadth of this unit. Try to answer each from memory before checking against the sections above.

1. Q1. Why does Kafka guarantee message order only within a single partition, and how does a producer ensure related messages land in the same partition?
2. Q2. What two things must happen atomically for Spark's Kafka integration to achieve exactly-once processing, and what additional requirement does the SINK have to meet?
3. Q3. What specific problem does Change Data Capture (CDC) solve that direct application instrumentation cannot, without modifying the source system?

☒ Answer key (cover before peeking)

A1: Ordering across an entire topic would require full coordination across every partition on every write, defeating the parallelism partitions exist to provide. A producer ensures related messages land together by supplying the same KEY (e.g., a user ID) — Kafka hashes the key to deterministically choose a partition, so all messages with that key always go to the same one.

A2: Writing the processed output to the sink and advancing the checkpointed offset must happen atomically together — so a mid-batch failure replays exactly the unprocessed range. The sink must additionally be IDEMPOTENT (writing the same result twice has the same effect as once) for the end-to-end guarantee to actually hold; Spark's checkpointing alone isn't sufficient if the sink itself isn't idempotent.

A3: CDC captures every database change (insert/update/delete) directly from the database's own transaction log, requiring ZERO modification to the source application's code — unlike direct instrumentation, which requires deliberately adding event-producing code to the application itself, something teams are often unwilling or unable to do for legacy or business-critical systems.

Hands-On Exercise

Complete, end to end, using your local Kafka installation (Section 3) and a Spark Structured Streaming job:

4. Create a topic with 3 partitions and produce at least 15 JSON-formatted test messages (e.g., simulated order events with a customer ID, product, and amount), using at least 4 distinct customer IDs as message keys.
5. Start a console consumer with `--from-beginning` and confirm all 15 messages arrive, then use --describe` to note which partition each customer ID's messages consistently land in.`
6. Write a Spark Structured Streaming job that reads from this topic, parses the JSON using an explicit schema (Section 4.2), and computes a running total amount per customer, written to the console.

7. Add a second, independent consumer group (Section 5.3/6.2) — a simple console consumer is fine — reading the SAME topic, and confirm it receives all messages independently of the Spark job's own progress.
8. Finally, write one paragraph identifying which delivery semantic (Section 4.4) your Spark job is using by default, and what would happen to your running totals if the job crashed and restarted mid-stream.

 What this exercise is really testing

This single exercise deliberately exercises every syllabus topic in this unit at once: topic/partition fundamentals (Section 1), a working local installation (Section 3), the producer/consumer model with key-based partitioning (Section 5), fan-out to multiple independent consumers (Section 6), Spark's Kafka integration with schema parsing and aggregation (Section 4), and a concrete understanding of delivery semantics (Section 4.4) — a complete, realistic mini streaming pipeline built entirely from this unit's material.